

04 - Pymongo CRUD

CSCI 112 - Contemporary Databases

Maiko Francisco V Rugeria, MTM



Learning Objectives

- Perform basic MongoDB CRUD using Pymongo

1

Scenario for the Day



Context Setting

- Rural Savings would like to start their digital transformation with PRIME, their core banking system for retail credit card accounts.

2

PyMongo CRUD



Guided Learning

Go to our course github page and access the folder lecture_supplements/04-pymongoCRUD. Follow the [README.md](#) instructions.



Code Structure

This code is broken down into different python scripts, one for each of the CRUD functionalities. They are named according to their functions:

- create.py - For insert/create functions
- read.py - For read/query functions
- update.py - For update functions
- delete.py - For delete functions

There are 2 other python scripts: utils.py and main.py. utils.py contains functions that establish and destroy connections to mongoDB. main.py is our main python code where all function calls are made in sequence.

It was designed this way for brevity. This is how I want you to design your code for your final project.



Code Structure

Always watch out for comments that look like these:

```
""
```

```
this is a comment
```

```
""
```

or

```
# this is another comment
```

as they would give you information on what is being done or instructions on what to do next. Comments marked with TODO needs action from your side in order to complete the code.



Initialization (utils.py)

Import the necessary libraries into your Python code.

pymongo – library that allows connection and different functions related to MongoDB.

```
from pymongo import *
```

or

```
import pymongo
```

What's the difference between the 2 import methods?

from pymongo import * - function usage does not require using dot notation.
Import pymongo - requires dot notation in function usage.

Example: (Connecting to a MongoDB Database)

MongoClient('localhost',27017) VS pymongo.MongoClient('localhost',27017)



Initialization (utils.py)

```
def openConnection(host='localhost') :  
    port = 27017  
    conn = MongoClient(host,port)  
    return conn
```

Create function to connect to MongoDB.

This code block declares a function named `openConnection()` that creates a connection to a mongoDB database at host `localhost` port `27017`. The host is parameterized so that you can pass a value on function call in case you want to connect to a db not in `localhost`.



Initialization (utils.py)

```
def closeConnection(conn):  
    conn.close()
```

Create function to close the MongoDB Connection.

This code block declares the function `closeConnection()` that accepts a `mongoDB` connection in a variable `conn` as its parameter. This will close the `conn` connection to the MongoDB database. It is highly recommended to close connections that are not in use to prevent any issues with the MongoDB database.



Initialization (main.py)

```
conn = openConnection()  
db = conn['myDB']  
  
<MongoDB operations here...>  
  
closeConnection(conn)
```

Use the `conn` variable to open the MongoDB connection and interact with the database.

This code block opens a connection and stores it into the `conn` variable. It is then used to refer to `myDB` database. You may now refer to collections inside `myDB` using `db["<collection name>"]`.

When all operations are completed, the `conn` variable is passed to the `closeConnection` function to close the connection.



PyMongo Insert (create.py)

Open your resource folder and locate the file create.py

This Python file contains function declarations that perform creation of records in your MongoDB database.

These are run in the main.py script. Follow the instructions inside the script (in comments) to determine the proper way of running the script.



PyMongo Read (Query)

Open your resource folder and locate the file read.py

This Python file contains function declarations that perform queries in your MongoDB database.

These are run in the main.py script. Follow the instructions inside the script (in comments) to determine the proper way of running the script.



Pymongo find_one vs find

Printing find_one shows results:

```
kiko@Kikos-MacBook-Air ruralSavings % python3 main.py
{'_id': 0, 'account': {'accountNumber': 'A872010018X', 'firstName': 'Michellin', 'lastName': 'Star', 'homeBranch': 'BGC', 'cardNumber': '1982039481279823'}, 'transactionDate': datetime.datetime(2020, 2, 19, 0, 0), 'transactionType': 'debit', 'amount': 246817, 'vendor': 'Fitness Last Pioneer'}
```

Printing find shows a cursor that can be iterated through to get results:

```
kiko@Kikos-MacBook-Air ruralSavings % python3 main.py
<pymongo.synchronous.cursor.Cursor object at 0x103ad3b80>
```



PyMongo Update

Scenario

In 2019, Rural Savings was targeted by a massive cybersecurity attack. All of their 2019 data in ruralSavingsPrime.transactions, unfortunately, were tampered with. Particularly, 2019 debit transaction vendors were replaced with incorrect values.

Since the data is now fraudulent, the leadership decided to let it go since they didn't have any backups.

Task: For all debit transactions in 2019, replace the value of “type” to “Fraud”, add a field “isFraud” with a value of True, and rename the “vendor” field to “fraudVendor”.



PyMongo Update

Open your resource folder and locate the file `update.py`

This Python file contains function declarations that perform updating of records in your MongoDB database.

These are run in the `main.py` script. Follow the instructions inside the script (in comments) to determine the proper way of running the script.



PyMongo Update

```
kiko@Kikos-MacBook-Air ruralSavings % python3 main.py
UpdateResult({'n': 78, 'nModified': 78, 'ok': 1.0, 'updatedExisting': True}, acknowledged=True)
kiko@Kikos-MacBook-Air ruralSavings %
```

Running an update command shows you the following:

- Count of retrieved documents
- Count of modified documents
- Status (ok: 1.0 means the update was successful)



PyMongo Delete

Scenario

Why keep your fraud data that is generally useless. It just reminds you of your past failures.

Task: Delete all fraud data that were compromised in the 2019 cybersecurity attack.



PyMongo Delete

Open your resource folder and locate the file delete.py

This Python file contains function declarations that perform deleting of records from your MongoDB database.

These are run in the main.py script. Follow the instructions inside the script (in comments) to determine the proper way of running the script.



PyMongo Delete

```
kiko@Kikos-MacBook-Air ruralSavings % python3 main.py  
DeleteResult({'n': 78, 'ok': 1.0}, acknowledged=True)  
kiko@Kikos-MacBook-Air ruralSavings %
```

Running a delete command shows you the following:

- Count of deleted documents
- Status (ok: 1.0 means the update was successful)

End.

CSCI 112 - Contemporary Databases

Maiko Francisco V Rugeria, MTM